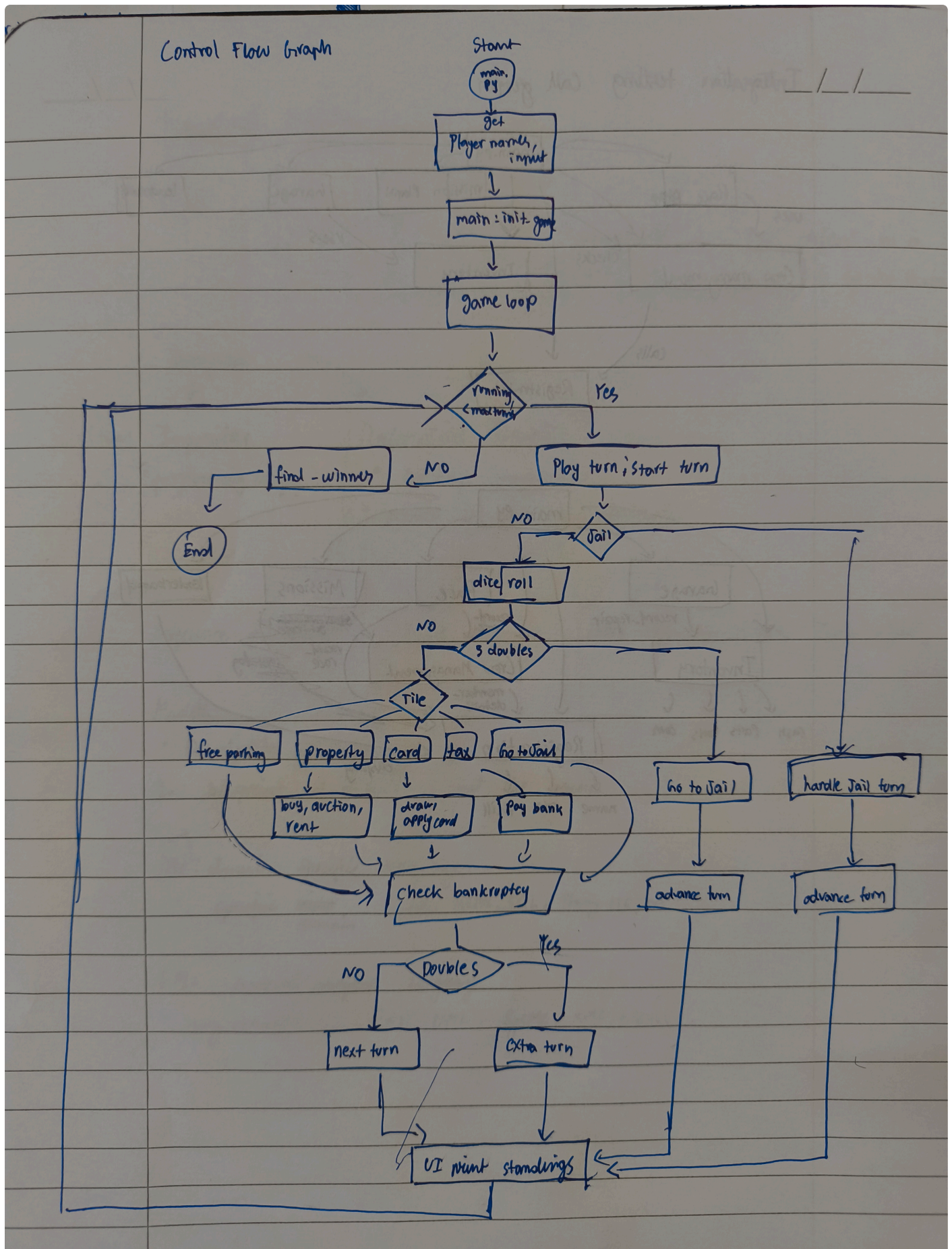


Whitebox testing

Control Flow Graph



```
***** Module config
moneypoly/config.py:1:0: C0114: Missing module docstring
(missing-module-docstring)
***** Module dice
moneypoly/dice.py:1:0: C0114: Missing module docstring
(missing-module-docstring)
moneypoly/dice.py:2:0: E0401: Unable to import
'moneypoly.config' (import-error)
moneypoly/dice.py:25:12: W0201: Attribute 'doubles_streak'
defined outside __init__ (attribute-defined-outside-init)
moneypoly/dice.py:2:0: W0611: Unused BOARD_SIZE imported from
moneypoly.config (unused-import)
***** Module cards
moneypoly/cards.py:4:0: C0301: Line too long (111/100) (line-
too-long)
moneypoly/cards.py:5:0: C0301: Line too long (112/100) (line-
too-long)
moneypoly/cards.py:6:0: C0301: Line too long (111/100) (line-
too-long)
moneypoly/cards.py:7:0: C0301: Line too long (112/100) (line-
too-long)
moneypoly/cards.py:8:0: C0301: Line too long (113/100) (line-
too-long)
moneypoly/cards.py:9:0: C0301: Line too long (113/100) (line-
too-long)
moneypoly/cards.py:10:0: C0301: Line too long (113/100) (line-
too-long)
moneypoly/cards.py:11:0: C0301: Line too long (112/100) (line-
too-long)
moneypoly/cards.py:12:0: C0301: Line too long (111/100) (line-
too-long)
moneypoly/cards.py:13:0: C0301: Line too long (112/100) (line-
too-long)
moneypoly/cards.py:14:0: C0301: Line too long (112/100) (line-
too-long)
moneypoly/cards.py:15:0: C0301: Line too long (120/100) (line-
too-long)
moneypoly/cards.py:19:0: C0301: Line too long (113/100) (line-
too-long)
```

```
moneypoly/cards.py:20:0: C0301: Line too long (112/100) (line-  
too-long)  
moneypoly/cards.py:21:0: C0301: Line too long (112/100) (line-  
too-long)  
moneypoly/cards.py:22:0: C0301: Line too long (111/100) (line-  
too-long)  
moneypoly/cards.py:23:0: C0301: Line too long (113/100) (line-  
too-long)  
moneypoly/cards.py:24:0: C0301: Line too long (112/100) (line-  
too-long)  
moneypoly/cards.py:25:0: C0301: Line too long (112/100) (line-  
too-long)  
moneypoly/cards.py:26:0: C0301: Line too long (113/100) (line-  
too-long)  
moneypoly/cards.py:27:0: C0301: Line too long (113/100) (line-  
too-long)  
moneypoly/cards.py:28:0: C0301: Line too long (112/100) (line-  
too-long)  
moneypoly/cards.py:29:0: C0301: Line too long (112/100) (line-  
too-long)  
moneypoly/cards.py:30:0: C0301: Line too long (111/100) (line-  
too-long)  
moneypoly/cards.py:1:0: C0114: Missing module docstring  
(missing-module-docstring)  
***** Module bank  
moneypoly/bank.py:1:0: C0114: Missing module docstring  
(missing-module-docstring)  
moneypoly/bank.py:2:0: E0401: Unable to import  
'moneypoly.config' (import-error)  
moneypoly/bank.py:5:0: C0115: Missing class docstring (missing-  
class-docstring)  
moneypoly/bank.py:1:0: W0611: Unused import math (unused-  
import)  
***** Module game  
moneypoly/game.py:453:0: C0325: Unnecessary parens after 'not'  
keyword (superfluous-parens)  
moneypoly/game.py:462:0: C0325: Unnecessary parens after 'not'  
keyword (superfluous-parens)  
moneypoly/game.py:468:0: C0304: Final newline missing (missing-  
final-newline)  
moneypoly/game.py:1:0: C0114: Missing module docstring  
(missing-module-docstring)
```

```
moneypoly/game.py:3:0: E0401: Unable to import
'moneypoly.config' (import-error)
moneypoly/game.py:12:0: E0401: Unable to import
'moneypoly.player' (import-error)
moneypoly/game.py:13:0: E0401: Unable to import
'moneypoly.board' (import-error)
moneypoly/game.py:14:0: E0401: Unable to import
'moneypoly.bank' (import-error)
moneypoly/game.py:15:0: E0401: Unable to import
'moneypoly.dice' (import-error)
moneypoly/game.py:16:0: E0401: Unable to import
'moneypoly.cards' (import-error)
moneypoly/game.py:17:0: E0401: Unable to import 'moneypoly'
(import-error)
moneypoly/game.py:20:0: R0902: Too many instance attributes
(9/7) (too-many-instance-attributes)
moneypoly/game.py:295:4: R0912: Too many branches (15/12) (too-
many-branches)
moneypoly/game.py:381:28: W1309: Using an f-string that does
not have any interpolated variables (f-string-without-
interpolation)
moneypoly/game.py:402:12: R1723: Unnecessary "elif" after
"break", remove the leading "el" from "elif" (no-else-break)
moneypoly/game.py:1:0: W0611: Unused import os (unused-import)
moneypoly/game.py:3:0: W0611: Unused GO_TO_JAIL_POSITION
imported from moneypoly.config (unused-import)
***** Module board
moneypoly/board.py:1:0: C0114: Missing module docstring
(missing-module-docstring)
moneypoly/board.py:1:0: E0401: Unable to import
'moneypoly.property' (import-error)
moneypoly/board.py:2:0: E0401: Unable to import
'moneypoly.config' (import-error)
moneypoly/board.py:108:11: C0121: Comparison 'prop.is_mortgaged
== True' should be 'prop.is_mortgaged is True' if checking for
the singleton value True, or 'prop.is_mortgaged' if testing for
truthiness (singleton-comparison)
***** Module ui
moneypoly/ui.py:1:0: C0114: Missing module docstring (missing-
module-docstring)
moneypoly/ui.py:69:4: W0702: No exception type(s) specified
(bare-except)
```

```
***** Module property
moneypoly/property.py:1:0: C0114: Missing module docstring
(missing-module-docstring)
moneypoly/property.py:1:0: R0902: Too many instance attributes
(9/7) (too-many-instance-attributes)
moneypoly/property.py:6:4: R0913: Too many arguments (6/5)
(too-many-arguments)
moneypoly/property.py:6:4: R0917: Too many positional arguments
(6/5) (too-many-positional-arguments)
moneypoly/property.py:48:8: R1705: Unnecessary "else" after
"return", remove the "else" and de-indent the code inside it
(no-else-return)
moneypoly/property.py:64:0: C0115: Missing class docstring
(missing-class-docstring)
***** Module player
moneypoly/player.py:87:0: C0304: Final newline missing
(missing-final-newline)
moneypoly/player.py:1:0: C0114: Missing module docstring
(missing-module-docstring)
moneypoly/player.py:2:0: E0401: Unable to import
'moneypoly.config' (import-error)
moneypoly/player.py:5:0: R0902: Too many instance attributes
(8/7) (too-many-instance-attributes)
moneypoly/player.py:45:8: W0612: Unused variable 'old_position'
(unused-variable)
moneypoly/player.py:1:0: W0611: Unused import sys (unused-
import)

-----
Your code has been rated at 8.17/10
```

change 1

added docstring to config

change 2

added docstring to dice

removed unused variable import BOARD_SIZE

changed double_streak to init initialization

change 3

add a doc string

reduced line size

change 4

added a bank.py doc string

added Bank class doc string

removed math import as it was unused

created init.py file to fix the import error

change 5

fixed unnecessary paras after not key word in game.py

change 6

added doc string to game.py

added final new line to game.py

change 7

fixed import errors in game.py

change 8

fixed too many attributes under game class init in game.py

change 9

fixed too many branches by adding multiple helper functions

change 10

fixed unused OS import

fixed unused go_to_jail_position

fixed the fstirng

removed elif after break statement

change 11

added docstring board.py

fixed import errors

fixed singleton comparision error

change 12

added docstring to ui.py

fixed a try except caluse

change 13

added docstring to property.py

fixed too many attribute errors by removing unused self.houses , and
converting mortagage calculation into a method

change 14

removed unecesary else in property.py

added doc string for class

change 15

fixed all issues in player

pylint for main.py

```
main.py:1:0: C0114: Missing module docstring (missing-module-  
docstring)
```

```
main.py:4:0: C0116: Missing function or method docstring  
(missing-function-docstring)
```

```
main.py:11:0: C0116: Missing function or method docstring  
(missing-function-docstring)
```

```
-----
```

```
---
```

```
Your code has been rated at 8.24/10 (previous run: 8.24/10,  
+0.00)
```

change 16

added docstrings to main.py

white box test cases

result from pytest

```
===== test session starts
=====
platform linux -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir:
/home/anivarth/college/sem4/dass/as2/dass_as2/moneypoly/moneypoly
plugins: cov-7.1.0
collected 111 items
```

```
test_moneypoly.py
```

```
F..F....F.....F.....F.....F.F.... [ 49%]
FFF.F....F.F.....F....FFFF.....FFF.....F.....
[100%]
```

```
===== FAILURES
```

```
=====
```

```
_____ test_1_1_standard_movement_and_passing_go
```

```
player = Player('TestPlayer', balance=1500, pos=2)
```

```
def test_1_1_standard_movement_and_passing_go(player):
    player.position = 38
    initial_balance = player.balance
    new_pos = player.move(4)
    assert new_pos == 2
    assert player.position == 2
> assert player.balance == initial_balance + GO_SALARY
E     AssertionError: assert 1500 == (1500 + 200)
E         + where 1500 = Player('TestPlayer', balance=1500,
pos=2).balance
```

```
test_moneypoly.py:28: AssertionError
```

```
_____ test_2_1_voluntary_fine_payment
```

```
game = <moneypoly.game.Game object at 0x7f961defad50>
```

```
def test_2_1_voluntary_fine_payment(game):
    player = game.players[0]
    player.go_to_jail()
    initial_balance = player.balance
```

```

        with patch('moneypoly.ui.confirm', return_value=True),
        patch('builtins.input', return_value='s'):
            game._handle_jail_turn(player)
            assert player.in_jail is False
            assert player.jail_turns == 0
>            assert player.balance == initial_balance -
JAIL_FINE
E            AssertionError: assert 1500 == (1500 - 50)
E            + where 1500 = Player('Player1', balance=1500,
pos=18).balance

test_moneypoly.py:61: AssertionError
----- Captured stdout call -----
-----
Player1 is in jail (turn 1/3).
Player1 paid the $50 fine and is released.
Player1 rolled: 4 + 4 = 8 (DOUBLES)
Player1 moved to position 18 [property]
Tennessee Avenue is unowned – asking price $180.
Player1 passes on Tennessee Avenue.
_____ test_3_3_paying_rent_with_group_multiplier
_____

game = <moneypoly.game.Game object at 0x7f961d738e20>

def test_3_3_paying_rent_with_group_multiplier(game):
    p1, p2 = game.players[0],
game.board.get_property_at(39)
    prop2 = game.board.get_property_at(37) # Park Place

    p2.owner = p1
    prop2.owner = p1 # P1 owns the Dark Blue group

    player_b = game.players[1]
    initial_balance = player_b.balance

    game.pay_rent(player_b, p2) # Base rent 50, double is
100

    assert player_b.balance == initial_balance - 100
>    assert p1.balance == 1500 + 100
E    AssertionError: assert 1500 == (1500 + 100)

```

```
E         + where 1500 = Player('Player1', balance=1500,
pos=0).balance
```

```
test_moneypoly.py:122: AssertionError
```

```
----- Captured stdout call -----
-----
```

```
Player2 paid $100 rent on Boardwalk to Player1.
```

```
_____ test_16_1_get_player_names_blanks
_____
```

```
def test_16_1_get_player_names_blanks():
>     from moneypoly.main import get_player_names
E     ModuleNotFoundError: No module named 'moneypoly.main'
```

```
test_moneypoly.py:340: ModuleNotFoundError
```

```
_____ test_19_1_bank_negative_collect_and_payout
_____
```

```
def test_19_1_bank_negative_collect_and_payout():
    from moneypoly.bank import Bank
    from moneypoly.player import Player
    bank = Bank()
    initial_funds = bank.get_balance()
    bank.collect(-50)
>     assert bank.get_balance() == initial_funds
E     assert 20530 == 20580
E         + where 20530 = get_balance()
E         + where get_balance = Bank(funds=20530).get_balance
```

```
test_moneypoly.py:442: AssertionError
```

```
_____ test_21_5_game_play_turn_luxury_tax
_____
```

```
game = <moneypoly.game.Game object at 0x7f961dac2450>
```

```
def test_21_5_game_play_turn_luxury_tax(game):
    p = game.players[0]
    p.position = 36 # chance, but we want to land on luxury
tax (38)
    with patch.object(game.dice, 'roll', return_value=2), \
         patch.object(game.dice, 'is_doubles',
return_value=False):
```

```

        game.play_turn()
>         assert p.balance == 1500 - 100 # luxury tax is 100
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E         AssertionError: assert 1425 == (1500 - 100)
E         + where 1425 = Player('Player1', balance=1425,
pos=38).balance

```

```

test_moneypoly.py:522: AssertionError
----- Captured stdout call -----
-----

```

```

=====
Turn 1 | Player1 | $1500
=====
Player1 rolled: 0 + 0 = 0
Player1 moved to position 38 [luxury_tax]
Player1 paid luxury tax: $75.
_____ test_21_7_game_play_turn_railroad
_____

```

```

game = <moneypoly.game.Game object at 0x7f961d803c50>

```

```

def test_21_7_game_play_turn_railroad(game):
    p = game.players[0]
    p.position = 3 # land on RR (5)
    with patch.object(game.dice, 'roll', return_value=2), \
         patch.object(game.dice, 'is_doubles',
return_value=False), \
         patch('builtins.input', return_value="s"): # Skip
buying
        game.play_turn()
>         assert game.board.get_property_at(5).owner is None
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E         AttributeError: 'NoneType' object has no attribute
'owner'

```

```

test_moneypoly.py:539: AttributeError
----- Captured stdout call -----
-----

```

```

=====
Turn 1 | Player1 | $1500

```

```
=====
Player1 rolled: 0 + 0 = 0
Player1 moved to position 5 [railroad]
_____ test_22_1_trade_cash_transfer
_____
```

```
game = <moneypoly.game.Game object at 0x7f961dac33d0>
```

```
def test_22_1_trade_cash_transfer(game):
    p1, p2 = game.players[0], game.players[1]
    prop = game.board.get_property_at(39)
    prop.owner = p1
    p1.add_property(prop)

    initial_p1 = p1.balance
    initial_p2 = p2.balance

    # p2 buys from p1 for 500
    game.trade(p1, p2, prop, 500)

>     assert p1.balance == initial_p1 + 500
E     AssertionError: assert 1500 == (1500 + 500)
E         + where 1500 = Player('Player1', balance=1500,
pos=0).balance
```

```
test_moneypoly.py:594: AssertionError
```

```
----- Captured stdout call -----
-----
```

```
Trade complete: Player1 sold Boardwalk to Player2 for $500.
```

```
_____ test_22_2_trade_negative_cash
_____
```

```
game = <moneypoly.game.Game object at 0x7f961d8e4350>
```

```
def test_22_2_trade_negative_cash(game):
    p1, p2 = game.players[0], game.players[1]
    prop = game.board.get_property_at(39)
    prop.owner = p1
    p1.add_property(prop)

    # Trade shouldn't accept negative cash
>     assert game.trade(p1, p2, prop, -100) is False
```

^^

test_moneypoly.py:604:

```
-----  
-----  
moneypoly/game.py:211: in trade  
    buyer.deduct_money(cash_amount)  
-----  
-----
```

```
self = Player('Player2', balance=1500, pos=0), amount = -100  
  
    def deduct_money(self, amount):  
        """Deduct funds from this player's balance. Amount must  
be non-negative."""  
        if amount < 0:  
>             raise ValueError(f"Cannot deduct a negative amount:  
{amount}")  
E             ValueError: Cannot deduct a negative amount: -100
```

```
moneypoly/player.py:58: ValueError  
_____ test_22_3_trade_no_properties_massive_cash  
_____
```

game = <moneypoly.game.Game object at 0x7f961db979d0>

```
def test_22_3_trade_no_properties_massive_cash(game):  
    p1, p2 = game.players[0], game.players[1]  
    p2.balance = 50000  
    # Try trading None  
>    assert game.trade(p1, p2, None, 50000) is False  
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

test_moneypoly.py:610:

```
-----  
-----  
  
self = <moneypoly.game.Game object at 0x7f961db979d0>  
seller = Player('Player1', balance=1500, pos=0)  
buyer = Player('Player2', balance=50000, pos=0), prop = None,  
cash_amount = 50000
```

```

def trade(self, seller, buyer, prop, cash_amount):
    """
    Execute a property trade: `seller` transfers `prop` to
    `buyer`
    in exchange for `cash_amount` from `buyer`.
    Returns True on success.
    """

```

```

> if prop.owner != seller:

```

```

    ^^^^^^^^^^^^^

```

```

E AttributeError: 'NoneType' object has no attribute
'owner'

```

```

moneyopoly/game.py:204: AttributeError

```

```

_____ test_22_5_trade_and_immediate_rent
_____

```

```

game = <moneyopoly.game.Game object at 0x7f961d848bd0>

```

```

def test_22_5_trade_and_immediate_rent(game):

```

```

    p1, p2 = game.players[0], game.players[1]

```

```

    prop = game.board.get_property_at(39)

```

```

    prop.owner = p1

```

```

    p1.add_property(prop)

```

```

    game.trade(p1, p2, prop, 50)

```

```

    initial_p1 = p1.balance

```

```

    game.pay_rent(p1, prop)

```

```

> assert p1.balance == initial_p1 - 50

```

```

E AssertionError: assert 1400 == (1500 - 50)

```

```

E + where 1400 = Player('Player1', balance=1400,
pos=0).balance

```

```

test_moneyopoly.py:632: AssertionError

```

```

----- Captured stdout call -----
-----

```

```

    Trade complete: Player1 sold Boardwalk to Player2 for $50.

```

```

    Player1 paid $100 rent on Boardwalk to Player2.

```

```

_____ test_23_1_jail_roll_doubles_release
_____

```

```

game = <moneyopoly.game.Game object at 0x7f961d919750>

```

```

def test_23_1_jail_roll_doubles_release(game):
    p1 = game.players[0]
    p1.go_to_jail()
    with patch.object(game.dice, 'is_doubles',
return_value=True), \
        patch.object(game.dice, 'roll', return_value=8):
        # Code handles card and fine, but skips doubles
        logic
        # If I patch builtins.input it will bypass
        properties
        with patch('moneypoly.ui.confirm',
return_value=False), patch('builtins.input', return_value='s'):
            game._handle_jail_turn(p1)
>         assert p1.in_jail is False
E         AssertionError: assert True is False
E         + where True = Player('Player1', balance=1500,
pos=10).in_jail

```

```

test_moneypoly.py:670: AssertionError
----- Captured stdout call -----
-----
    Player1 is in jail (turn 1/3).
_____ test_23_3_dice_initial_doubles_state
_____

```

```

def test_23_3_dice_initial_doubles_state():
    from moneypoly.dice import Dice
    d = Dice()
    # Before roll, it computes 0 == 0
>     assert d.is_doubles() is False
E     assert True is False
E     + where True = is_doubles()
E     + where is_doubles = Dice(die1=0, die2=0,
streak=0).is_doubles

```

```

test_moneypoly.py:688: AssertionError
_____ test_23_10_consecutive_jail
_____

```

```

game = <moneypoly.game.Game object at 0x7f961d72cf50>

```

```

def test_23_10_consecutive_jail(game):
    p1 = game.players[0]
    p1.go_to_jail()
    p1.jail_turns = 1
    # Next turn p1 uses card, rolls doubles 3 times, goes
back to jail
    game.dice.doubles_streak = 3
    with patch('moneypoly.ui.confirm', return_value=True),
patch('builtins.input', return_value='s'):
        game._handle_jail_turn(p1)
>     assert p1.in_jail is True
E     AssertionError: assert False is True
E         + where False = Player('Player1', balance=1500,
pos=19).in_jail

```

test_moneypoly.py:744: AssertionError

----- Captured stdout call -----

```

Player1 is in jail (turn 2/3).
Player1 paid the $50 fine and is released.
Player1 rolled: 4 + 5 = 9
Player1 moved to position 19  [property]
New York Avenue is unowned – asking price $200.
Player1 passes on New York Avenue.

```

```

test_24_net_worth_variations[True-1700]

```

```

game = <moneypoly.game.Game object at 0x7f961d9b3cd0>, mtg =
True
exp_net_worth = 1700

```

```

@pytest.mark.parametrize("mtg, exp_net_worth", [
    (True, 1500 + 200),  # test_24_7
    (False, 1000 + 400), # test_24_9
])
def test_24_net_worth_variations(game, mtg, exp_net_worth):
    p1 = game.players[0]
    bw = game.board.get_property_at(39)
    bw.owner = p1
    p1.properties.append(bw)
    if mtg:
        bw.mortgage()

```

```

        else:
            p1.balance = 1000
>         assert p1.net_worth() == exp_net_worth
E         AssertionError: assert 1500 == 1700
E         +   where 1500 = net_worth()
E         +   where net_worth = Player('Player1', balance=1500,
pos=0).net_worth

```

```

test_moneypoly.py:791: AssertionError
_____ test_24_net_worth_variations[False-1400]
_____

```

```

game = <moneypoly.game.Game object at 0x7f961d84a5d0>, mtg =
False
exp_net_worth = 1400

```

```

@pytest.mark.parametrize("mtg, exp_net_worth", [
    (True, 1500 + 200), # test_24_7
    (False, 1000 + 400), # test_24_9
])
def test_24_net_worth_variations(game, mtg, exp_net_worth):
    p1 = game.players[0]
    bw = game.board.get_property_at(39)
    bw.owner = p1
    p1.properties.append(bw)
    if mtg:
        bw.mortgage()
    else:
        p1.balance = 1000
>         assert p1.net_worth() == exp_net_worth
E         AssertionError: assert 1000 == 1400
E         +   where 1000 = net_worth()
E         +   where net_worth = Player('Player1', balance=1000,
pos=0).net_worth

```

```

test_moneypoly.py:791: AssertionError
_____ test_24_8_buy_property_exact_change
_____

```

```

game = <moneypoly.game.Game object at 0x7f961dfef7d0>

```

```

def test_24_8_buy_property_exact_change(game):

```

```

        p1 = game.players[0]
        prop = game.board.get_property_at(39)
        p1.balance = 400
>         assert game.buy_property(p1, prop) is True
E         AssertionError: assert False is True
E         +   where False = buy_property(Player('Player1',
balance=400, pos=0), Property('Boardwalk', pos=39,
owner='unowned'))
E         +   where buy_property = <moneypoly.game.Game object
at 0x7f961dfef7d0>.buy_property

```

test_moneypoly.py:797: AssertionError

----- Captured stdout call -----

Player1 cannot afford Boardwalk (\$400).

_____ test_25_1_transfer_all_insufficient_funds

game = <moneypoly.game.Game object at 0x7f961d72c150>

```

def test_25_1_transfer_all_insufficient_funds(game):
    p1, p2, p3 = game.players
    p2.balance = 5
    game._card_action_transfer_all(p1, 10)
    # p2 should go bankrupt in correct rules, but code just
silently skips
>         assert p2.balance == -5
E         AssertionError: assert 5 == -5
E         +   where 5 = Player('Player2', balance=5,
pos=0).balance

```

test_moneypoly.py:808: AssertionError

_____ test_25_10_recursive_chance_to_chest

game = <moneypoly.game.Game object at 0x7f961da26150>

```

def test_25_10_recursive_chance_to_chest(game):
    p1 = game.players[0]
    # Chance card sends to community chest position (2)
    with patch.object(game.decks["community"], "draw",
return_value={"action": "collect", "value": 50, "description":

```

```

    ""}):
        game._card_action_move_to(p1, 2)
>         assert p1.balance == 1500 + 50
E         AssertionError: assert 1500 == (1500 + 50)
E         +   where 1500 = Player('Player1', balance=1500,
pos=2).balance

```

```

test_moneypoly.py:869: AssertionError
_____ test_26_1_bank_negative_collected
_____

```

```

def test_26_1_bank_negative_collected():
    from moneypoly.bank import Bank
    bank = Bank()
    bank.collect(-10)
>     assert bank._total_collected == 0
E     assert -10 == 0
E     +   where -10 = Bank(funds=20570)._total_collected

```

```

test_moneypoly.py:877: AssertionError
_____ test_26_2_bank_negative_loans
_____

```

```

game = <moneypoly.game.Game object at 0x7f961dcb8fd0>

```

```

def test_26_2_bank_negative_loans(game):
    bank = game.bank
    p = game.players[0]
    bank._funds = 100
    bank.give_loan(p, 500)
>     assert bank.get_balance() == -400
E     assert 100 == -400
E     +   where 100 = get_balance()
E     +   where get_balance = Bank(funds=100).get_balance

```

```

test_moneypoly.py:884: AssertionError
----- Captured stdout call -----
-----
    Bank issued a $500 emergency loan to Player1.
_____ test_27_1_bank_coverage
_____

```

```

def test_27_1_bank_coverage():
    from moneypoly.bank import Bank
    from moneypoly.player import Player
    b = Bank()
    p = Player("A")
    assert b.pay_out(-10) == 0
    b.give_loan(p, -50)
    b.give_loan(p, 500)
    assert b.total_loans_issued() == 500
    b.summary()
> assert repr(b) == "Bank(funds=20030)"
E     AssertionError: assert 'Bank(funds=20580)' ==
'Bank(funds=20030)'
E
E         - Bank(funds=20030)
E         ?                  ^^
E         + Bank(funds=20580)
E         ?                  ^^

```

test_moneypoly.py:950: AssertionError

----- Captured stdout call -----

```

Bank issued a $500 emergency loan to A.
Bank reserves   : $20,580
Total collected: $0
Loans issued    : 1 ($500)

```

===== short test summary info
=====

FAILED

test_moneypoly.py::test_1_1_standard_movement_and_passing_go -
AssertionError: assert 1500 == (1500 + 200)

FAILED test_moneypoly.py::test_2_1_voluntary_fine_payment -
AssertionError: assert 1500 == (1500 - 50)

FAILED

test_moneypoly.py::test_3_3_paying_rent_with_group_multiplier -
AssertionError: assert 1500 == (1500 + 100)

FAILED test_moneypoly.py::test_16_1_get_player_names_blanks -
ModuleNotFoundError: No module named 'moneypoly.main'

FAILED

test_moneypoly.py::test_19_1_bank_negative_collect_and_payout -
assert 20530 == 20580

FAILED test_moneypoly.py::test_21_5_game_play_turn_luxury_tax -

```

AssertionError: assert 1425 == (1500 - 100)
FAILED test_moneypoly.py::test_21_7_game_play_turn_railroad -
AttributeError: 'NoneType' object has no attribute 'owner'
FAILED test_moneypoly.py::test_22_1_trade_cash_transfer -
AssertionError: assert 1500 == (1500 + 500)
FAILED test_moneypoly.py::test_22_2_trade_negative_cash -
ValueError: Cannot deduct a negative amount: -100
FAILED
test_moneypoly.py::test_22_3_trade_no_properties_massive_cash -
AttributeError: 'NoneType' object has no attribute 'owner'
FAILED test_moneypoly.py::test_22_5_trade_and_immediate_rent -
AssertionError: assert 1400 == (1500 - 50)
FAILED test_moneypoly.py::test_23_1_jail_roll_doubles_release -
AssertionError: assert True is False
FAILED test_moneypoly.py::test_23_3_dice_initial_doubles_state
- assert True is False
FAILED test_moneypoly.py::test_23_10_consecutive_jail -
AssertionError: assert False is True
FAILED test_moneypoly.py::test_24_net_worth_variations[True-
1700] - AssertionError: assert 1500 == 1700
FAILED test_moneypoly.py::test_24_net_worth_variations[False-
1400] - AssertionError: assert 1000 == 1400
FAILED test_moneypoly.py::test_24_8_buy_property_exact_change -
AssertionError: assert False is True
FAILED
test_moneypoly.py::test_25_1_transfer_all_insufficient_funds -
AssertionError: assert 5 == -5
FAILED test_moneypoly.py::test_25_10_recursive_chance_to_chest
- AssertionError: assert 1500 == (1500 + 50)
FAILED test_moneypoly.py::test_26_1_bank_negative_collected -
assert -10 == 0
FAILED test_moneypoly.py::test_26_2_bank_negative_loans -
assert 100 == -400
FAILED test_moneypoly.py::test_27_1_bank_coverage -
AssertionError: assert 'Bank(funds=20580)' ==
'Bank(funds=20030)'
===== 22 failed, 89 passed in 1.07s
=====

```

error 1

fixed issue where go payment was only given if landed on go , now it is given

whenever player passes go

error 2

fixed issue where paying fine didnt deduct from the palyers account

error 3

fixed issue where paying rent wasnt credited to the property owner

error 4

main.py was causing import errors due to being outside the directory, fixed by keeping inside the directory

error 5

added fix to ignore negative amounts in bank balance

error 6

fixed the issue where railroads were missing from purchasable items

error 7

when buying property the cash wasnt being deposited into the sellers account

error 8

fixed bug where if no property object was passed during trade , trade was still accepted

error 9

fixed bug where calculating rent for properties owned by a owner used any instead of all

error 10

fixed bug where provision to roll doubles to leave jail wasnt implemented

error 11

updated dice randomized roll range to be from 1-6 instead of 1-5

fixed error where is_doubles incorrectly returned true for a new dice , reset dice because they were initialized to 0,0

error 12

fixed error where trade method was allowing negative amounts

error 13

fixed error where 3 double streaks makes the player go to jail

error 14

networth calculation only returned cash balance, instead of overall cash and property value

error 15

fixed bug where player had to have more money than price to buy it, but now can have more than or equal to the price to buy it

error 16

fixed bug in transfer all function where players with low funds didnt get money deducted

error 17

fixed issue where chance , community chest wasnt handled when a player was forcibly moved to a new position

error 18

fixed issue where banks balance was never decreased when giving money to the player

some of the test case errors were resultant from the same bug , hence only 18 error fixes in total

error 19

fixed issue where interactive menu is not being called

error 20

fixed issue where cards werent being shuffled

```
Dataview (inline field '=====
test session starts =====
platform linux -- Python 3.14.3, pytest-9.0.2, pluggy-
1.6.0
rootdir:
/home/anivarth/college/sem4/dass/as2/dass_as2/moneypoly
/moneypoly
plugins: cov-7.1.0
collected 111 items
```

```
test_moneypoly.py
```

```
F..F....F.....F.....F.....F.F....
[ 49%]
FFF.F....F.F.....F....FFFF.....FFF.....F.....
.
[100%]
```

```
===== FAILURES
=====
```

```
test_1_1_standard_movement_and_passing_go
```

```
player = Player('TestPlayer', balance=1500, pos=2)
```

```
def
```

```
test_1_1_standard_movement_and_passing_go(player):
```

```
    player.position = 38
```

```
    initial_balance = player.balance
```

```
    new_pos = player.move(4)
```

```
    assert new_pos == 2
```

```
    assert player.position == 2
```

```
>    assert player.balance == initial_balance +
```

```
GO_SALARY
```

```
E           AssertionError: assert 1500 == (1500 + 200)
E           +   where 1500 = Player('TestPlayer',
balance=1500, pos=2).balance
```

```
test_moneypoly.py:28: AssertionError
```

```
test_2_1_voluntary_fine_payment
```

```
game = <moneypoly.game.Game object at 0x7f961defad50>
```

```
def test_2_1_voluntary_fine_payment(game):
    player = game.players[0]
    player.go_to_jail()
    initial_balance = player.balance

    with patch('moneypoly.ui.confirm',
return_value=True), patch('builtins.input',
return_value='s'):
        game._handle_jail_turn(player)
        assert player.in_jail is False
        assert player.jail_turns == 0
>         assert player.balance == initial_balance -
JAIL_FINE
E           AssertionError: assert 1500 == (1500 - 50)
E           +   where 1500 = Player('Player1',
balance=1500, pos=18).balance
```

```
test_moneypoly.py:61: AssertionError
```

```
----- Captured stdout call -----
-----
```

```
Player1 is in jail (turn 1/3).
Player1 paid the $50 fine and is released.
Player1 rolled: 4 + 4 = 8 (DOUBLES)
Player1 moved to position 18 [property]
Tennessee Avenue is unowned – asking price $180.
Player1 passes on Tennessee Avenue.
```

`test_3_3_paying_rent_with_group_multiplier`

```
game = <moneypoly.game.Game object at 0x7f961d738e20>
```

```
def
test_3_3_paying_rent_with_group_multiplier(game):
    p1, p2 = game.players[0],
game.board.get_property_at(39)
    prop2 = game.board.get_property_at(37) # Park
Place

    p2.owner = p1
    prop2.owner = p1 # P1 owns the Dark Blue group

    player_b = game.players[1]
    initial_balance = player_b.balance

    game.pay_rent(player_b, p2) # Base rent 50,
double is 100
    assert player_b.balance == initial_balance -
100
>    assert p1.balance == 1500 + 100
E    AssertionError: assert 1500 == (1500 + 100)
E        + where 1500 = Player('Player1',
balance=1500, pos=0).balance
```

```
test_moneypoly.py:122: AssertionError
```

```
----- Captured stdout call -----
-----
```

```
    Player2 paid $100 rent on Boardwalk to Player1.
```

`test_16_1_get_player_names_blanks`

```
def test_16_1_get_player_names_blanks():
```

```
>         from moneypoly.main import get_player_names
E         ModuleNotFoundError: No module named
'moneypoly.main'
```

```
test_moneypoly.py:340: ModuleNotFoundError
```

```
test_19_1_bank_negative_collect_and_payout
```

```
def test_19_1_bank_negative_collect_and_payout():
    from moneypoly.bank import Bank
    from moneypoly.player import Player
    bank = Bank()
    initial_funds = bank.get_balance()
    bank.collect(-50)
>     assert bank.get_balance() == initial_funds
E     assert 20530 == 20580
E         + where 20530 = get_balance()
E         +       where get_balance =
Bank(funds=20530).get_balance
```

```
test_moneypoly.py:442: AssertionError
```

```
test_21_5_game_play_turn_luxury_tax
```

```
game = <moneypoly.game.Game object at 0x7f961dac2450>
```

```
def test_21_5_game_play_turn_luxury_tax(game):
    p = game.players[0]
    p.position = 36 # chance, but we want to land
on luxury tax (38)
    with patch.object(game.dice, 'roll',
return_value=2), \
        patch.object(game.dice, 'is_doubles',
return_value=False):
        game.play_turn()
```



```
attribute 'owner'
```

```
test_moneypoly.py:539: AttributeError
```

```
----- Captured stdout call -----  
-----
```

```
=====
```

```
Turn 1 | Player1 | $1500
```

```
=====
```

```
Player1 rolled: 0 + 0 = 0
```

```
Player1 moved to position 5 [railroad]
```

```
_____ test_22_1_trade_cash_transfer  
_____
```

```
game = <moneypoly.game.Game object at 0x7f961dac33d0>
```

```
def test_22_1_trade_cash_transfer(game):  
    p1, p2 = game.players[0], game.players[1]  
    prop = game.board.get_property_at(39)  
    prop.owner = p1  
    p1.add_property(prop)
```

```
    initial_p1 = p1.balance  
    initial_p2 = p2.balance
```

```
    # p2 buys from p1 for 500  
    game.trade(p1, p2, prop, 500)
```

```
>         assert p1.balance == initial_p1 + 500  
E         AssertionError: assert 1500 == (1500 + 500)  
E         + where 1500 = Player('Player1',  
balance=1500, pos=0).balance
```

```
test_moneypoly.py:594: AssertionError
```

```
----- Captured stdout call -----  
-----
```

```
Trade complete: Player1 sold Boardwalk to Player2 for
```

\$500.

test_22_2_trade_negative_cash

game = <moneypoly.game.Game object at 0x7f961d8e4350>

```
def test_22_2_trade_negative_cash(game):
    p1, p2 = game.players[0], game.players[1]
    prop = game.board.get_property_at(39)
    prop.owner = p1
    p1.add_property(prop)

    # Trade shouldn't accept negative cash
> assert game.trade(p1, p2, prop, -100) is False
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

test_moneypoly.py:604:

```
-----
-----
moneypoly/game.py:211: in trade
    buyer.deduct_money(cash_amount)
-----
-----
```

```
self = Player('Player2', balance=1500, pos=0), amount =
-100
```

```
def deduct_money(self, amount):
    """Deduct funds from this player's balance.
    Amount must be non-negative."""
    if amount < 0:
>         raise ValueError(f"Cannot deduct a negative
amount: {amount}")
E         ValueError: Cannot deduct a negative
amount: -100
```

moneypoly/player.py:58: ValueError

test_22_3_trade_no_properties_massive_cash

game = <moneypoly.game.Game object at 0x7f961db979d0>

```
def
test_22_3_trade_no_properties_massive_cash(game):
    p1, p2 = game.players[0], game.players[1]
    p2.balance = 50000
    # Try trading None
>    assert game.trade(p1, p2, None, 50000) is False
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

test_moneypoly.py:610:

```
self = <moneypoly.game.Game object at 0x7f961db979d0>
seller = Player('Player1', balance=1500, pos=0)
buyer = Player('Player2', balance=50000, pos=0), prop =
None, cash_amount = 50000
```

```
def trade(self, seller, buyer, prop, cash_amount):
    """
    Execute a property trade: `seller` transfers
`prop` to `buyer`
    in exchange for `cash_amount` from `buyer`.
    Returns True on success.
    """
```

```
>    if prop.owner != seller:
           ^^^^^^^^^^^^^
```

E AttributeError: 'NoneType' object has no attribute 'owner'

moneypoly/game.py:204: AttributeError

test_22_5_trade_and_immediate_rent

game = <moneypoly.game.Game object at 0x7f961d848bd0>

```
def test_22_5_trade_and_immediate_rent(game):
    p1, p2 = game.players[0], game.players[1]
    prop = game.board.get_property_at(39)
    prop.owner = p1
    p1.add_property(prop)

    game.trade(p1, p2, prop, 50)

    initial_p1 = p1.balance
    game.pay_rent(p1, prop)
>    assert p1.balance == initial_p1 - 50
E    AssertionError: assert 1400 == (1500 - 50)
E    + where 1400 = Player('Player1',
balance=1400, pos=0).balance
```

test_moneypoly.py:632: AssertionError

----- Captured stdout call -----

Trade complete: Player1 sold Boardwalk to Player2 for \$50.

Player1 paid \$100 rent on Boardwalk to Player2.

test_23_1_jail_roll_doubles_release

game = <moneypoly.game.Game object at 0x7f961d919750>

```
def test_23_1_jail_roll_doubles_release(game):
    p1 = game.players[0]
    p1.go_to_jail()
    with patch.object(game.dice, 'is_doubles',
return_value=True), \
```

```

        patch.object(game.dice, 'roll',
return_value=8):
        # Code handles card and fine, but skips
doubles logic
        # If I patch builtins.input it will bypass
properties
        with patch('moneypoly.ui.confirm',
return_value=False), patch('builtins.input',
return_value='s'):
            game._handle_jail_turn(p1)
>         assert p1.in_jail is False
E         AssertionError: assert True is False
E         + where True = Player('Player1',
balance=1500, pos=10).in_jail

```

```

test_moneypoly.py:670: AssertionError
----- Captured stdout call -----
-----
    Player1 is in jail (turn 1/3).

```

```

test_23_3_dice_initial_doubles_state

```

```

def test_23_3_dice_initial_doubles_state():
    from moneypoly.dice import Dice
    d = Dice()
    # Before roll, it computes 0 == 0
>     assert d.is_doubles() is False
E     assert True is False
E     + where True = is_doubles()
E     + where is_doubles = Dice(die1=0, die2=0,
streak=0).is_doubles

```

```

test_moneypoly.py:688: AssertionError
_____ test_23_10_consecutive_jail

```

```
game = <moneypoly.game.Game object at 0x7f961d72cf50>
```

```
def test_23_10_consecutive_jail(game):
    p1 = game.players[0]
    p1.go_to_jail()
    p1.jail_turns = 1
    # Next turn p1 uses card, rolls doubles 3
times, goes back to jail
    game.dice.doubles_streak = 3
    with patch('moneypoly.ui.confirm',
return_value=True), patch('builtins.input',
return_value='s'):
        game._handle_jail_turn(p1)
>         assert p1.in_jail is True
E         AssertionError: assert False is True
E         + where False = Player('Player1',
balance=1500, pos=19).in_jail
```

```
test_moneypoly.py:744: AssertionError
```

```
----- Captured stdout call -----
-----
```

```
Player1 is in jail (turn 2/3).
Player1 paid the $50 fine and is released.
Player1 rolled: 4 + 5 = 9
Player1 moved to position 19 [property]
New York Avenue is unowned – asking price $200.
Player1 passes on New York Avenue.
```

```
_____ test_24_net_worth_variations[True-
1700] _____
```

```
game = <moneypoly.game.Game object at 0x7f961d9b3cd0>,
mtg = True
exp_net_worth = 1700
```

```
@pytest.mark.parametrize("mtg, exp_net_worth", [
    (True, 1500 + 200), # test_24_7
    (False, 1000 + 400), # test_24_9
```

```

    ])
    def test_24_net_worth_variations(game, mtg,
exp_net_worth):
        p1 = game.players[0]
        bw = game.board.get_property_at(39)
        bw.owner = p1
        p1.properties.append(bw)
        if mtg:
            bw.mortgage()
        else:
            p1.balance = 1000
>         assert p1.net_worth() == exp_net_worth
E         AssertionError: assert 1500 == 1700
E         + where 1500 = net_worth()
E         +       where net_worth = Player('Player1',
balance=1500, pos=0).net_worth

```

```

test_moneypoly.py:791: AssertionError
_____ test_24_net_worth_variations[False-
1400] _____

```

```

game = <moneypoly.game.Game object at 0x7f961d84a5d0>,
mtg = False
exp_net_worth = 1400

```

```

@pytest.mark.parametrize("mtg, exp_net_worth", [
    (True, 1500 + 200), # test_24_7
    (False, 1000 + 400), # test_24_9
])
def test_24_net_worth_variations(game, mtg,
exp_net_worth):
    p1 = game.players[0]
    bw = game.board.get_property_at(39)
    bw.owner = p1
    p1.properties.append(bw)
    if mtg:
        bw.mortgage()

```

```

        else:
            p1.balance = 1000
>         assert p1.net_worth() == exp_net_worth
E         AssertionError: assert 1000 == 1400
E         +   where 1000 = net_worth()
E         +   where net_worth = Player('Player1',
balance=1000, pos=0).net_worth

```

test_moneypoly.py:791: AssertionError

test_24_8_buy_property_exact_change

game = <moneypoly.game.Game object at 0x7f961dfef7d0>

```

def test_24_8_buy_property_exact_change(game):
    p1 = game.players[0]
    prop = game.board.get_property_at(39)
    p1.balance = 400
>     assert game.buy_property(p1, prop) is True
E     AssertionError: assert False is True
E     +   where False =
buy_property(Player('Player1', balance=400, pos=0),
Property('Boardwalk', pos=39, owner='unowned'))
E     +   where buy_property = <moneypoly.game.Game
object at 0x7f961dfef7d0>.buy_property

```

test_moneypoly.py:797: AssertionError

```

----- Captured stdout call -----
-----
    Player1 cannot afford Boardwalk ($400).

```

test_25_1_transfer_all_insufficient_funds

game = <moneypoly.game.Game object at 0x7f961d72c150>

```

def
test_25_1_transfer_all_insufficient_funds(game):
    p1, p2, p3 = game.players
    p2.balance = 5
    game._card_action_transfer_all(p1, 10)
    # p2 should go bankrupt in correct rules, but
code just silently skips
>         assert p2.balance == -5
E         AssertionError: assert 5 == -5
E         + where 5 = Player('Player2', balance=5,
pos=0).balance

```

test_moneypoly.py:808: AssertionError

test_25_10_recursive_chance_to_chest

game = <moneypoly.game.Game object at 0x7f961da26150>

```

def test_25_10_recursive_chance_to_chest(game):
    p1 = game.players[0]
    # Chance card sends to community chest position
(2)
    with patch.object(game.decks["community"],
"draw", return_value={"action": "collect", "value": 50,
"description": ""}):
        game._card_action_move_to(p1, 2)
>         assert p1.balance == 1500 + 50
E         AssertionError: assert 1500 == (1500 + 50)
E         + where 1500 = Player('Player1',
balance=1500, pos=2).balance

```

test_moneypoly.py:869: AssertionError

test_26_1_bank_negative_collected

```

def test_26_1_bank_negative_collected():
    from moneypoly.bank import Bank
    bank = Bank()
    bank.collect(-10)
> assert bank._total_collected == 0
E assert -10 == 0
E + where -10 =
Bank(funds=20570)._total_collected

test_moneypoly.py:877: AssertionError
_____ test_26_2_bank_negative_loans
_____

```

```

game = <moneypoly.game.Game object at 0x7f961dcb8fd0>

```

```

def test_26_2_bank_negative_loans(game):
    bank = game.bank
    p = game.players[0]
    bank._funds = 100
    bank.give_loan(p, 500)
> assert bank.get_balance() == -400
E assert 100 == -400
E + where 100 = get_balance()
E + where get_balance =
Bank(funds=100).get_balance

test_moneypoly.py:884: AssertionError
----- Captured stdout call -----
-----
    Bank issued a $500 emergency loan to Player1.
_____ test_27_1_bank_coverage
_____

```

```

def test_27_1_bank_coverage():
    from moneypoly.bank import Bank
    from moneypoly.player import Player
    b = Bank()

```

```

        p = Player("A")
        assert b.pay_out(-10) == 0
        b.give_loan(p, -50)
        b.give_loan(p, 500)
        assert b.total_loans_issued() == 500
        b.summary()
>         assert repr(b) == "Bank(funds=20030)"
E         AssertionError: assert 'Bank(funds=20580)' ==
'Bank(funds=20030)'
E
E         - Bank(funds=20030)
E         ?                  ^^
E         + Bank(funds=20580)
E         ?                  ^^

```

test_moneypoly.py:950: AssertionError

----- Captured stdout call ----

```

Bank issued a $500 emergency loan to A.
Bank reserves   : $20,580
Total collected: $0
Loans issued    : 1 ($500)

```

===== short test summary info
=====

FAILED

test_moneypoly.py::test_1_1_standard_movement_and_passi
ng_go - AssertionError: assert 1500 == (1500 + 200)

FAILED

test_moneypoly.py::test_2_1_voluntary_fine_payment -
AssertionError: assert 1500 == (1500 - 50)

FAILED

test_moneypoly.py::test_3_3_paying_rent_with_group_mult
ipplier - AssertionError: assert 1500 == (1500 + 100)

FAILED

test_moneypoly.py::test_16_1_get_player_names_blanks -
ModuleNotFoundError: No module named 'moneypoly.main'

FAILED

```
test_moneypoly.py::test_19_1_bank_negative_collect_and_
payout - assert 20530 == 20580
FAILED
test_moneypoly.py::test_21_5_game_play_turn_luxury_tax
- AssertionError: assert 1425 == (1500 - 100)
FAILED
test_moneypoly.py::test_21_7_game_play_turn_railroad -
AttributeError: 'NoneType' object has no attribute
'owner'
FAILED test_moneypoly.py::test_22_1_trade_cash_transfer
- AssertionError: assert 1500 == (1500 + 500)
FAILED test_moneypoly.py::test_22_2_trade_negative_cash
- ValueError: Cannot deduct a negative amount: -100
FAILED
test_moneypoly.py::test_22_3_trade_no_properties_massiv
e_cash - AttributeError: 'NoneType' object has no
attribute 'owner'
FAILED
test_moneypoly.py::test_22_5_trade_and_immediate_rent -
AssertionError: assert 1400 == (1500 - 50)
FAILED
test_moneypoly.py::test_23_1_jail_roll_doubles_release
- AssertionError: assert True is False
FAILED
test_moneypoly.py::test_23_3_dice_initial_doubles_state
- assert True is False
FAILED test_moneypoly.py::test_23_10_consecutive_jail -
AssertionError: assert False is True
FAILED
test_moneypoly.py::test_24_net_worth_variations[True-
1700] - AssertionError: assert 1500 == 1700
FAILED
test_moneypoly.py::test_24_net_worth_variations[False-
1400] - AssertionError: assert 1000 == 1400
FAILED
test_moneypoly.py::test_24_8_buy_property_exact_change
- AssertionError: assert False is True
```

```

FAILED
test_moneypoly.py::test_25_1_transfer_all_insufficient_funds - AssertionError: assert 5 == -5
FAILED
test_moneypoly.py::test_25_10_recursive_chance_to_chest - AssertionError: assert 1500 == (1500 + 50)
FAILED
test_moneypoly.py::test_26_1_bank_negative_collected - assert -10 == 0
FAILED test_moneypoly.py::test_26_2_bank_negative_loans - assert 100 == -400
FAILED test_moneypoly.py::test_27_1_bank_coverage - AssertionError: assert 'Bank(funds=20580)' == 'Bank(funds=20030)'
===== 22 failed, 89 passed in 1.07s
====='): Error:
-- PARSING FAILED -----
-----

> 1 | ===== test session starts
=====
    | ^
    2 | platform linux -- Python 3.14.3, pytest-9.0.2,
pluggy-1.6.0
    3 | rootdir:
/home/anivarth/college/sem4/dass/as2/dass_as2/moneypoly
/moneypoly

```

Expected one of the following:

```

(' ', 'null', boolean, date, duration, file link, list
('[1, 2, 3]'), negated field, number, object ('{ a: 1,
b: 2 }'), string, variable

```